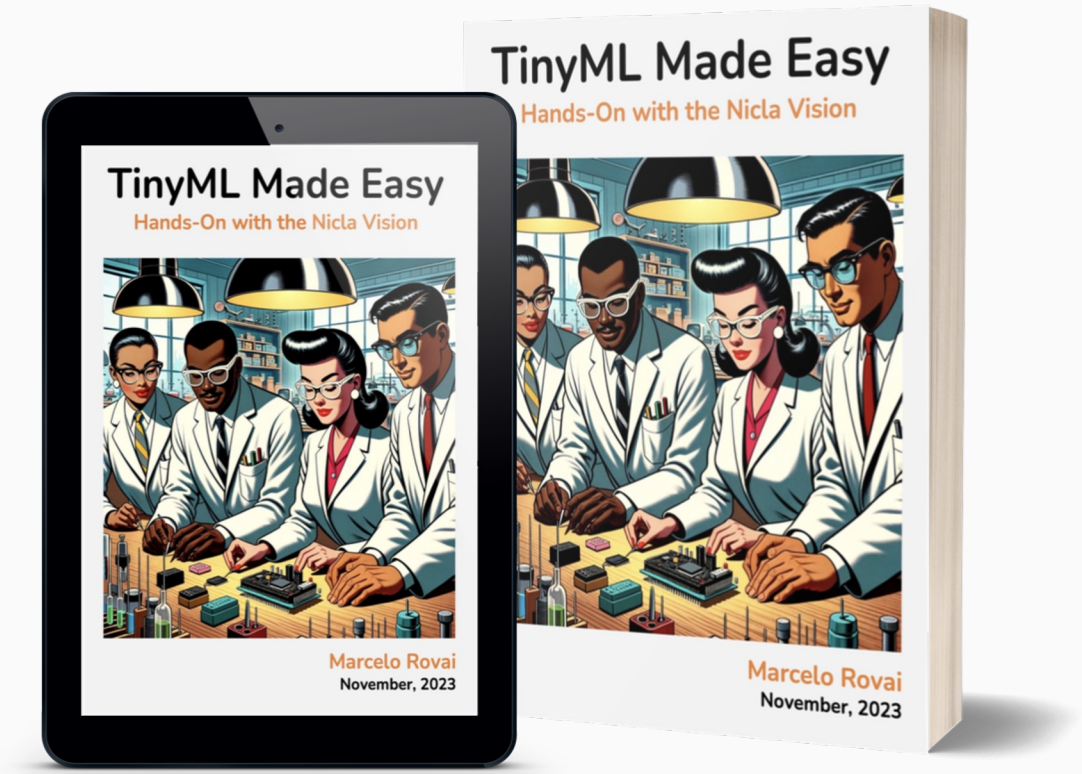
# Object detection

**Marco Zennaro, PhD**
The Abdus Salam International Centre for Theoretical Physics –Italy




https://mjrovai.github.io/TinyML_Made_Easy_NiclaV_eBook/

## Image Classification

(**Multi-Class** Classification)



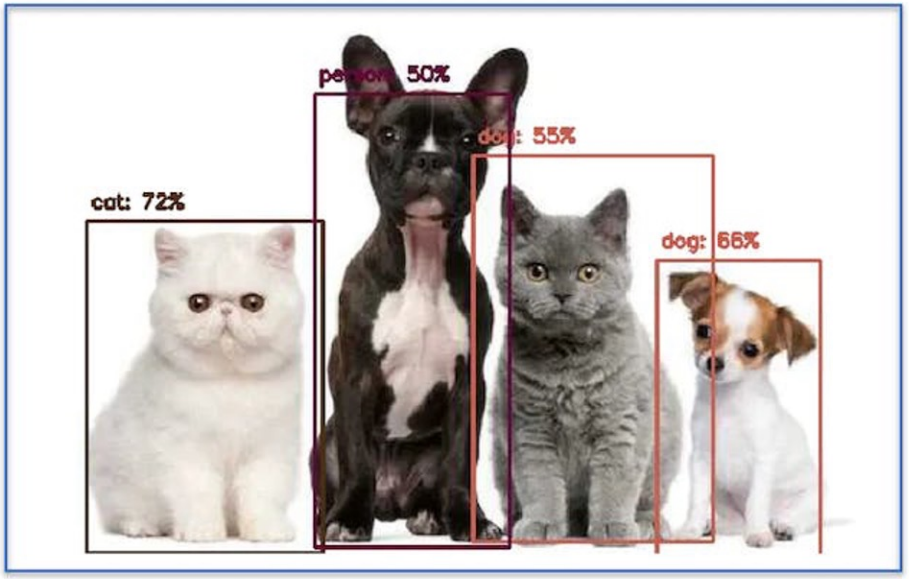
Cat: 70%



Dog: 80%

## Object Detection

**Multi-Label** Classification + Object **Localization**



# FOMO

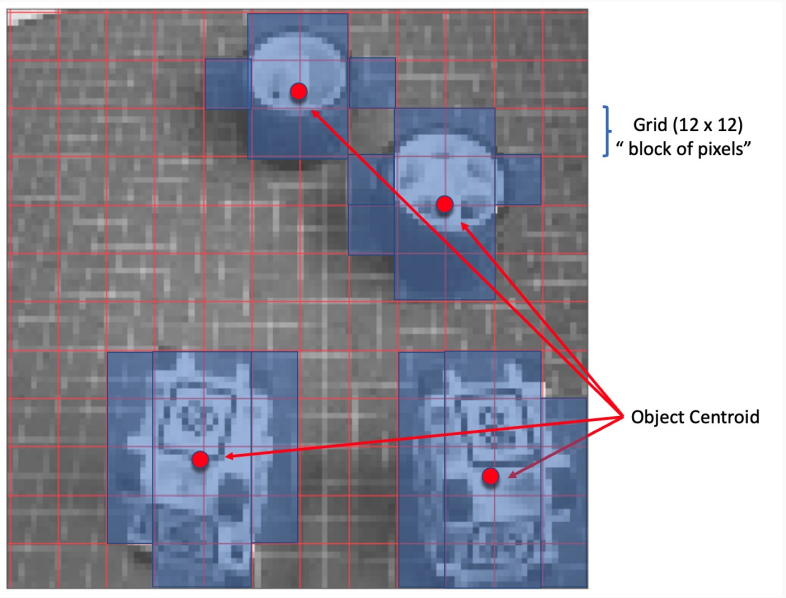
- Edge Impulse launched in 2022, **FOMO (Faster Objects, More Objects)**, a novel solution for performing object detection on embedded devices, not only on the Nicla Vision (Cortex M7) but also on Cortex M4F CPUs (Arduino Nano33 and OpenMV M4 series) and the Espressif ESP32 devices (ESP-CAM and XIAO ESP32S3 Sense).

- **We are interested in which object is in the image, its location (centroid), and how many we can find of it.**

- FOMO can use up to 30 times less energy and memory than traditional models like Mobilenet SSD and YOLOv5. FOMO can operate on microcontrollers with less than 200 KB of RAM. The main reason this is possible is that while other models calculate the object's size by drawing a square around it (bounding box), FOMO ignores the size of the image, providing only the information about where the object is located in the image, by means of its centroid coordinates.

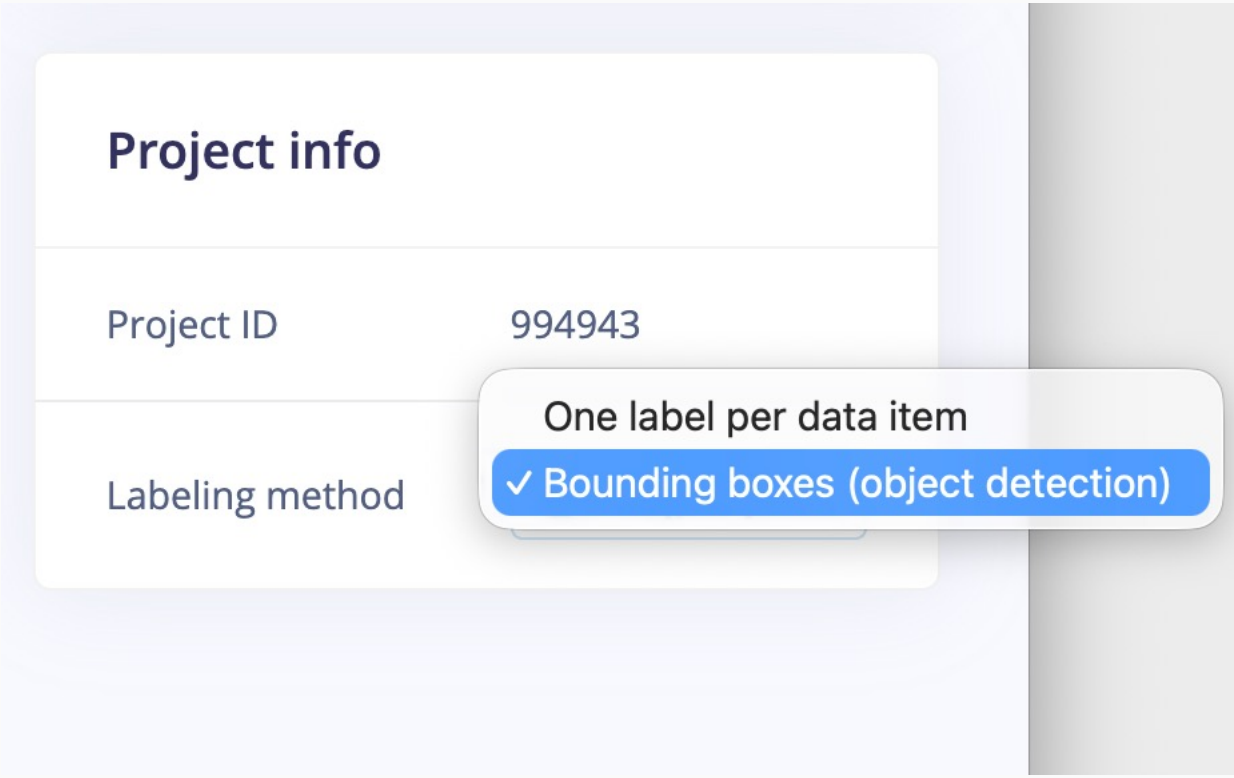
# FOMO

- FOMO takes the image in grayscale and divides it into blocks of pixels using a factor of 8. For the input of 96x96 , the grid would be 96/8=12 . Next, FOMO will run a classifier through each pixel block to calculate the probability that there is an object (box or a wheel for example) in each of them and, subsequently, determine the regions that have the highest probability of containing the object (If a pixel block has no objects, it will be classified as background). From the overlap of the final region, FOMO provides the coordinates of the centroid of this region.



# FOMO

- To use FOMO in Edge Impulse, on the Project Dashboard, go to Project info and select Bounding boxes (object detection), and at the right-top of the page, select Target, Arduino Nicla Vision (Cortex-M7).



# FOMO

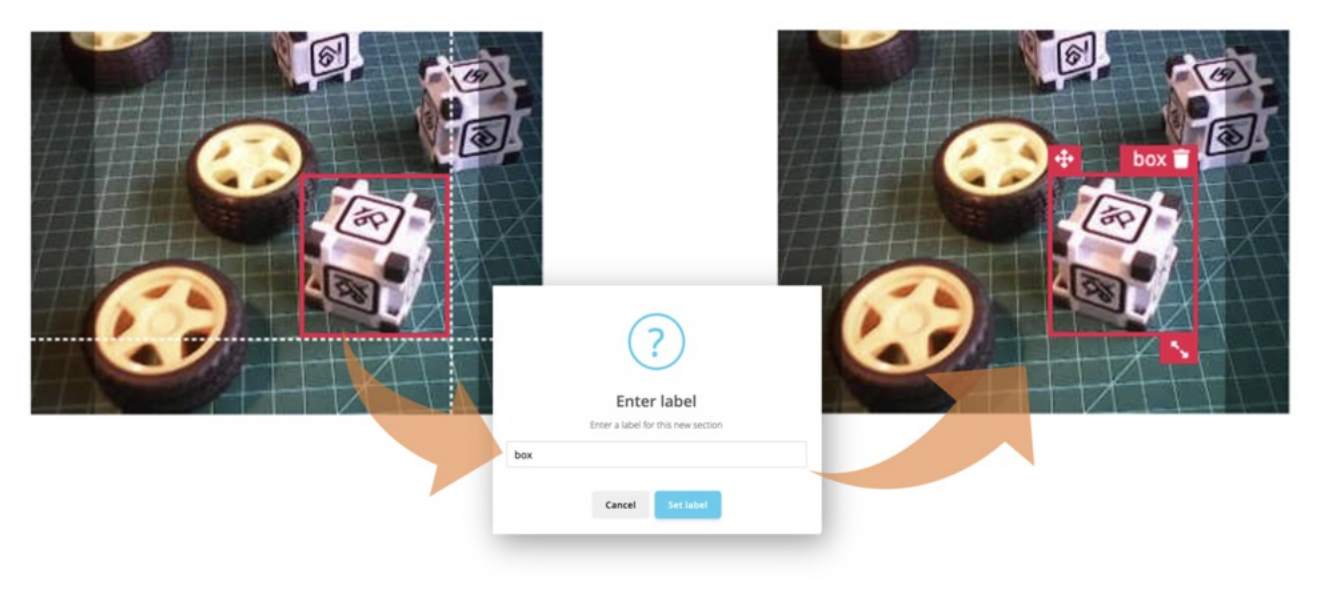
- On Egde Impulse, go to the Data acquisition tab, and on the UPLOAD DATA section, upload from your computer files captured.

- All the unlabeled images are uploaded, but they still need to be labeled appropriately before being used as a dataset in the project. The Studio has a tool for that purpose, which you can find in the link Labeling queue, in Data acquisition.

- Ordinary objects can quickly be identified and labeled using an existing library of pre-trained object detection models. In our case, the objects are not part of COCO datasets, we should select the option of tracking objects. With this option, once you draw bounding boxes and label the images in one frame, the objects will be tracked automatically from frame to frame, partially labeling the new ones (not all are correctly labeled).

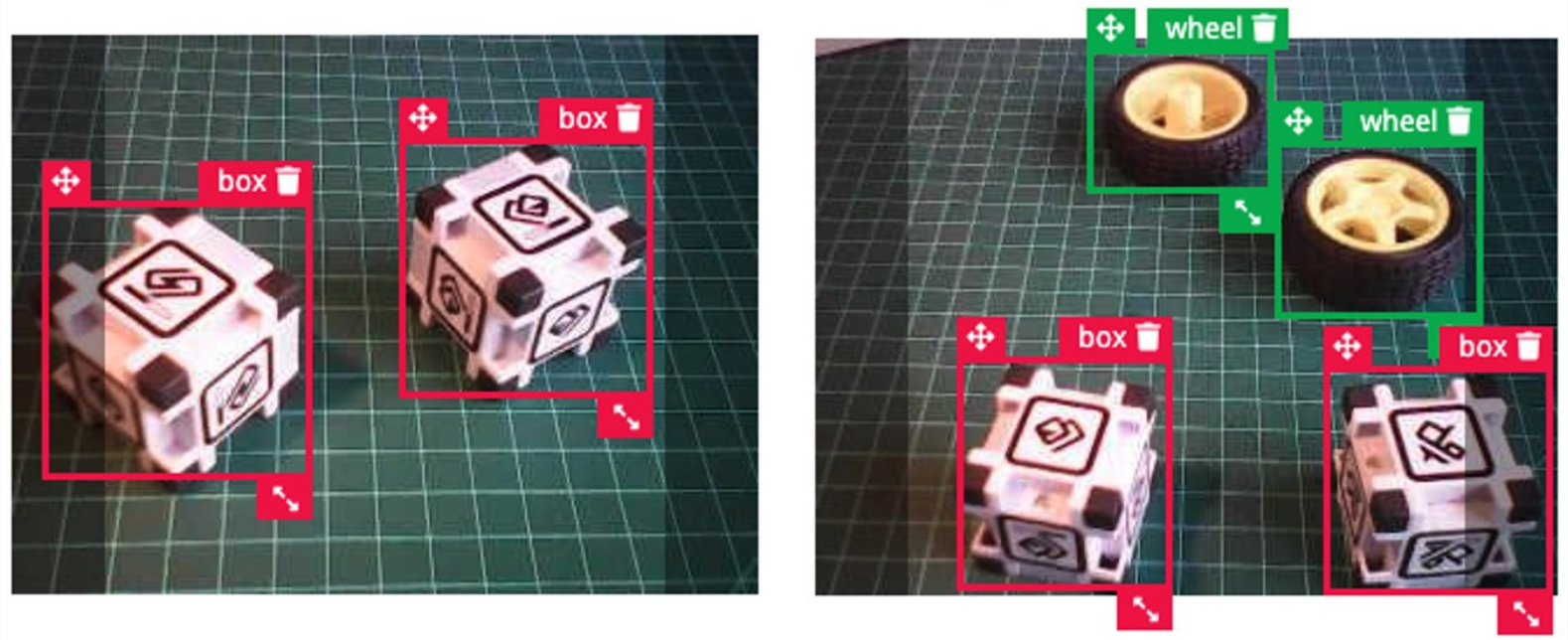
# FOMO

- Starting with the first image of your unlabeled data, use your mouse to drag a box around an object to add a label. Then click Save labels to advance to the next item.



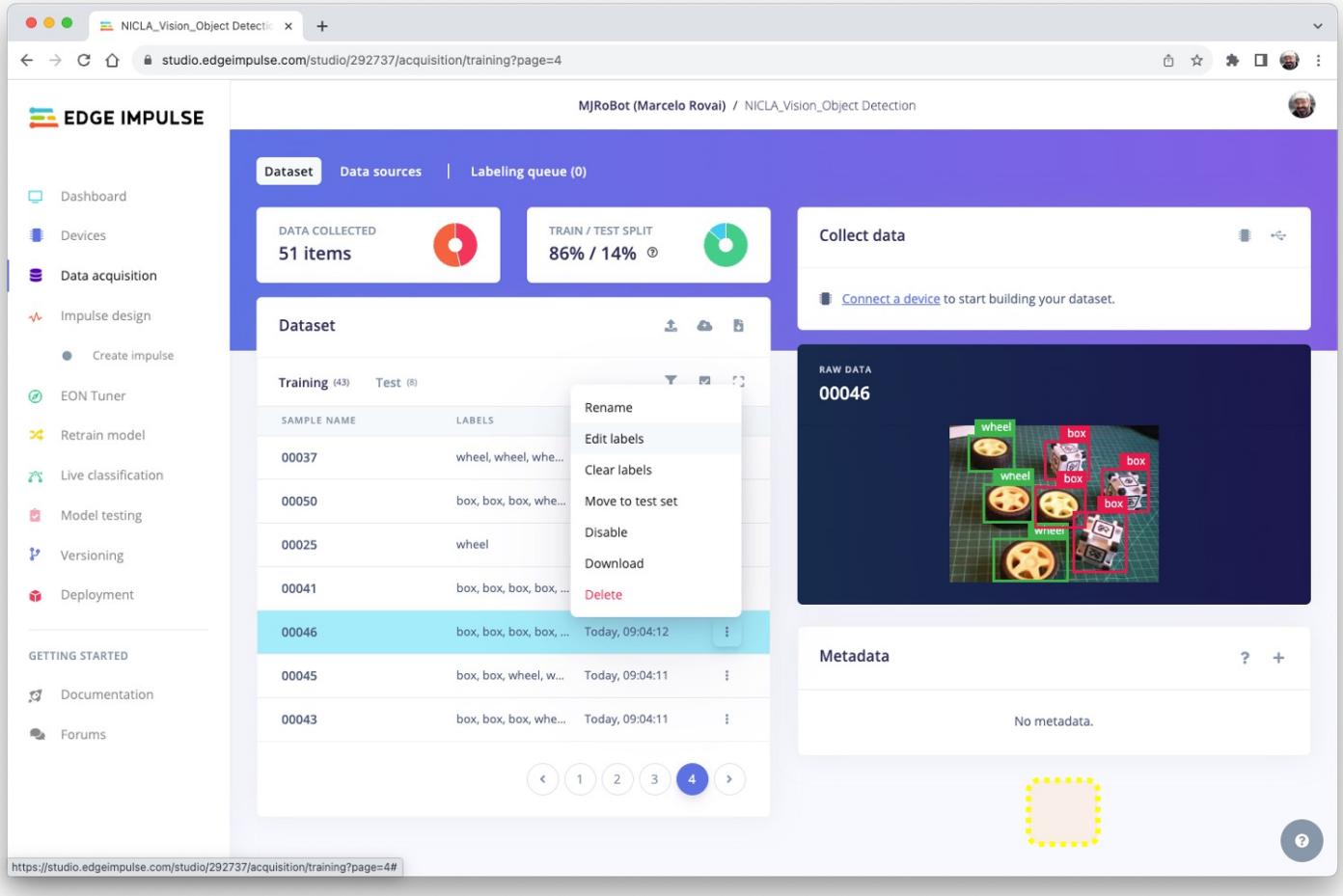
# FOMO

- Continue with this process until the queue is empty. At the end, all images should have the objects labeled as those samples below:



# FOMO

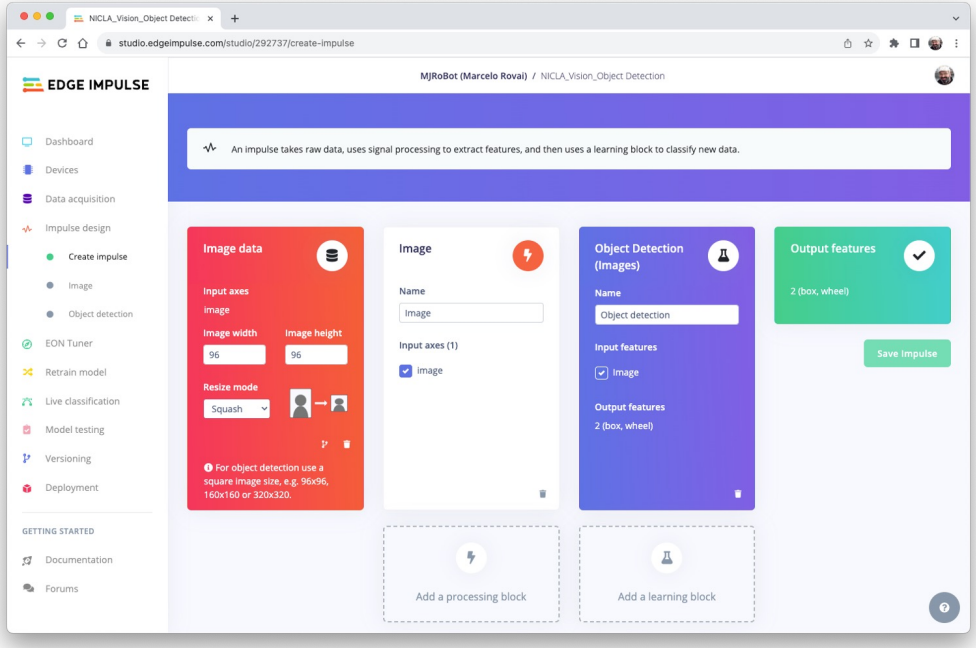
- Next, review the labeled samples on the Data acquisition tab. If one of the labels is wrong, it can be edited using the three dots menu after the sample name:



# FOMO

- Pre-processing consists of resizing the individual images and squashing them (squared form, without cropping). Afterward, the images are converted from RGB to Grayscale.

- When you design the model, select “Object Detection.”



# FOMO

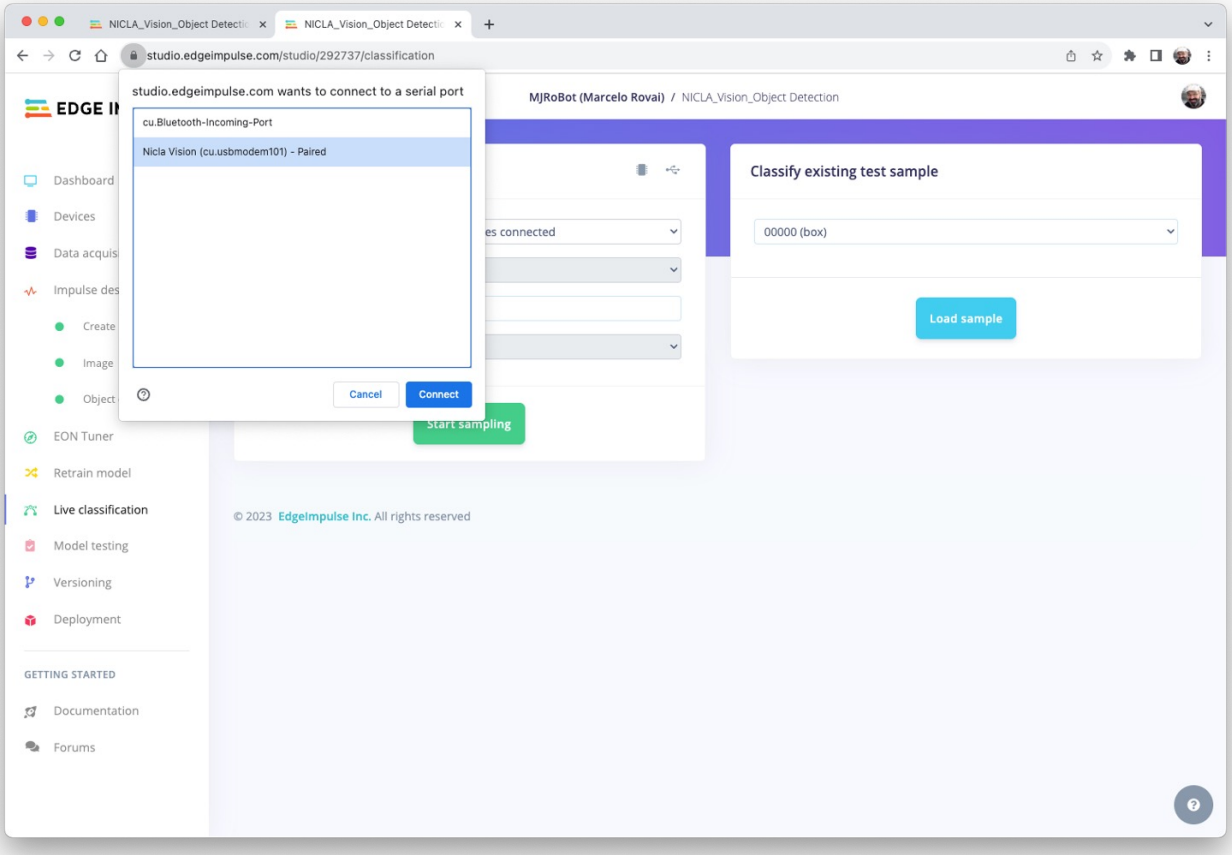
- For training, we should select a pre-trained model. Let's use the FOMO (Faster Objects, More Objects) MobileNetV2 0.35. This model uses around 250 KB of RAM and 80 KB of ROM (Flash), which suits well with our board since it has 1 MB of RAM and ROM.

- In object detection tasks, accuracy is generally not the primary evaluation metric. Object detection involves classifying objects and providing bounding boxes around them, making it a more complex problem than simple classification. The issue is that we do not have the bounding box, only the centroids. In short, using accuracy as a metric could be misleading and may not provide a complete understanding of how well the model is performing. Because of that, we will use the F1 score.

- The **F1 score** is the harmonic mean of precision (share of detections that are correct) and recall (share of real objects found): $F1 = 2 \cdot P \cdot R / (P + R)$. It ranges from 0 to 1 and is high only when the model makes few false detections and misses few objects.

# FOMO

- Since Edge Impulse officially supports the Nicla Vision, let’s connect it to the Studio for Live Classifciation.



# FOMO

- Train FOMO on cars and motorcycles. Both options work anywhere:

## Option A: public dataset

- Real-world street photos, already labeled with bounding boxes
- 5 classes: ambulance, bus, car, motorcycle, truck
- Keep only car and motorcycle when you import it into Edge Impulse

kaggle.com/datasets/abdallahwagih/cars-detection

## Option B: your own street

- Record 100+ images of traffic near you with the Nicla Vision or a phone
- Upload them to Edge Impulse and label each car and motorcycle
- Images from the place you deploy give the best accuracy there

# FOMO

- Challenge:
  - Label a dataset of cars and motorcycles (public or from your own street)
  - Train and deploy the model on the Nicla
  - Get outside and check if the system works in the real world
  - Send the number of cars and of motorcycles  (in the real world) via LoRa and visualize it via MQTT

